

Backend Fundamentals — Recap 6

curl internals, pagination proven, API versioning, and caching

Milestone: cursor pagination fully rebuilt and proven, the API is now versioned under /v1, and GET /v1/tasks has a working cache with correct invalidation on every write.

1. curl — Structure and Flags

curl ("Client URL") sends any HTTP request from the terminal — unlike a browser address bar, which only ever sends GET.

```
curl [flags] <URL>
```

```
-X POST          the HTTP method (defaults to GET if omitted)
-H "... "       adds a header, e.g. Content-Type or Idempotency-Key
-d '{...}'      the request body (for POST/PUT)
-i             include status line + response headers in the output
```

On Linux, JSON bodies can use single-quotes around the whole string with plain double quotes inside — no backslash-escaping needed, unlike Windows/MINGW.

```
curl -X POST http://localhost:3000/v1/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Buy eggs"}'
```

A bash loop for quickly seeding test data:

```
for i in $(seq 1 15); do
  curl -X POST http://localhost:3000/v1/tasks \
    -H "Content-Type: application/json" \
    -d "{\"title\": \"Task number $i\"}"
done
```

2. Idempotency Keys — Who Generates the Key, and When

Clarified a common confusion: the key is not something the server invents or matches across unrelated clicks. The **client** generates one key per logical attempt, at the moment an action starts, and reuses that exact same key only if it retries that same attempt (e.g. after a network timeout). A genuinely new click is a new attempt and gets a new key — correctly creating a new resource, since that really was the intent.

A real double-click (impatient clicking) is a separate problem, usually solved by disabling the button immediately after the first click — not by idempotency keys, which exist to make automatic client-side retries safe, not to stop deliberate repeat actions.

3. req.query With Multiple Parameters

Confirmed directly: multiple query parameters chain together with &, and all land in req.query as one object.

```
/tasks?after=5&limit=2
```

```
// req.query -> { after: '5', limit: '2' }
```

4. Cursor Pagination — Rebuilt and Directly Proven

```
app.get('/v1/tasks', (req, res) => {
  const after = req.query.after ? Number(req.query.after) : 0;
  const limit = 5;
  const tasks = db.prepare('SELECT * FROM tasks WHERE id > ? ORDER BY id LIMIT ?')
    .all(after, limit);
  res.send(tasks);
});
```

Proof performed directly, not just reasoned about: fetched a page ending near id 8, deleted id 10 (a task further ahead, not yet seen), then re-fetched with `?after=8`. Result: id 9 still appeared, id 10 was cleanly absent, and nothing was duplicated or silently skipped — unlike offset pagination, which counts positions that shift when rows are deleted.

Side discovery: `WHERE id > ?` still works correctly even if the row matching the cursor's own id was deleted — it's a pure numeric comparison, not a lookup of that specific row.

5. API Versioning

Changing an API's response shape directly (e.g. renaming a field) breaks every existing client the moment it ships. Fix: keep the old behavior under one URL prefix, and add new behavior under a new one, so both exist side by side.

```
app.get('/v1/tasks', ...); // existing clients keep getting { id, title }

app.get('/v2/tasks', (req, res) => {
  const tasks = db.prepare('SELECT * FROM tasks WHERE id > ? ORDER BY id LIMIT ?')
    .all(after, limit);
  const reshaped = tasks.map(t => ({ id: t.id, name: t.title }));
  res.send(reshaped); // new clients get { id, name }
});
```

Confirmed by testing: `/v1/tasks` returns title, `/v2/tasks` returns name, both reading from the exact same table. Versioning lives at the response-shaping layer, never in the database itself — which is exactly what makes running both versions simultaneously safe.

All existing CRUD routes were moved under `/v1/` as part of this.

6. Caching and Cache Invalidation

Caching means doing expensive work (a DB query) once and reusing the saved result for subsequent identical requests, skipping the database entirely — trading perfect freshness for speed.

```
let tasksCache = null;

app.get('/v1/tasks', (req, res) => {
  if (tasksCache) {
    console.log('Serving from cache');
    return res.send(tasksCache);
  }
  // ... database query logic ...
  tasksCache = tasks;
  res.send(tasks);
});
```

```
}
console.log('Cache miss – querying database');
const tasks = db.prepare('SELECT * FROM tasks WHERE id > ? ORDER BY id LIMIT ?')
    .all(after, limit);
tasksCache = tasks;
res.send(tasks);
});
```

The bug, demonstrated directly: after the cache filled, a new POST created a task correctly in the database, but GET kept serving the old cached snapshot — the new task was invisible until the cache was cleared. This is "cache invalidation" — the real difficulty is reliably catching every place data changes, not the caching mechanism itself.

Fix — clear the cache on every write:

```
tasksCache = null; // added after the insert/update/delete in POST, PUT, and DELETE
```

Verified for all three: POST, PUT, and DELETE each correctly force the next GET back to "Cache miss," and the response reflects the real, current data afterward.

7. Where the API Stands Now

- All routes versioned under /v1, with a /v2/tasks demonstrating a safe field rename
- Cursor-based pagination on GET /tasks, proven stable under deletion
- In-memory caching on GET /v1/tasks, with invalidation wired into POST/PUT/DELETE
- Idempotency keys, full CRUD, validation, error handling, and SQLite persistence all still in place from earlier

Next up: message queues & event-driven systems, load balancing & horizontal scaling, CAP theorem & replication — then the Phase 5 capstone.